

TaskFlow

Backend Geliştirme Proje Envanteri

Görev oluşturma, atama, durum ve öncelik takibiyle küçük ekiplerin iş akışını yöneten REST API ve bu API üzerinde çalışan responsive web arayüzünün eksiksiz teknik envanteri.

Node.js 18+

Express.js 5

JSON Dosya Kalıcılığı

Vanilla JS Frontend

Postman Test Paketi

Amaç ve Kapsam

TaskFlow, küçük yazılım ekiplerinin görev oluřturmasını, çalışanlara atamasını, durum ve önceliklerini takip etmesini saęlayan bir REST API ve bu API'yi kullanan responsive bir web arayüzüdür. Kimlik doğrulama veya kullanıcı giriři bulunmaz; ekip, doğrudan paylaşılan bir görev panosu üzerinden çalışır.

Senaryo

Bir ekip lideri yeni bir görev açar, açıklama ve öncelięini belirler ve bir çalışana atar. Çalışan görev durumunu güncelledikçe pano canlı olarak yenilenir; ekip lideri durum, öncelik, çalışan veya anahtar kelimeye göre filtreleyip özet raporlarla ilerlemeyi izler.

Temel Özellikler

- Görevler için tam CRUD işlemleri (oluřturma, listeleme, detay, güncelleme, silme)
- Duruma ve öncelięe göre filtreleme
- Başlık ve açıklamada büyük/küçük harf duyarsız anahtar kelime araması
- Çalışana göre görev listeleme (büyük/küçük harf duyarsız)
- Oluřturulma tarihine göre artan/azalan sıralama
- Sayfalama (`page` , `limit`)
- Tamamlanan, bekleyen ve genel özet raporları
- Tüm istekler için method, endpoint, durum kodu ve süreyi kaydeden logger
- `POST` / `PUT` gövdeleri için sunucu taraflı doğrulama

Mimari ve Kullanıcı Deneyimi

Backend Mimarisi

Uygulama, Express.js 5 üzerine katmanlı bir mimariyle kurulmuştur: `routes` istekleri ilgili controller'a yönlendirir, `controllers` iş mantığını yürütür, `middleware` katmanı doğrulama ve hata yönetimini üstlenir, `utils` katmanı ise dosya erişimi ve sorgu motorunu barındırır. Bu ayırım, her sorumluluğun tek bir yerde yönetilmesini sağlar.

JSON Dosyası Kalıcılığı

Veritabanı yerine `data/tasks.json` dosyası kullanılır. Dosya erişimi yalnızca `src/utils/file.js` üzerinden `readTasks()` / `writeTasks()` fonksiyonlarıyla yapılır; hiçbir route veya controller doğrudan dosya sistemine erişmez. Yalnızca başarılı `POST`, `PUT` ve `DELETE` işlemlerinden sonra dosyaya yazılır; listeleme, arama ve rapor istekleri dosyayı değiştirmez. Sunucu yeniden başlatıldığında veriler korunur.

Frontend Dashboard

Aynı Express uygulaması, `express.static('public')` ile derleme aracı veya framework gerektirmeyen vanilla HTML/CSS/JavaScript tabanlı bir dashboard sunar. Arayüz, tüm verileri `fetch()` ile REST API'den alır ve istatistik kartları, filtreleme araçları, görev kartları ile oluşturma/düzenleme/silme akışlarını tek bir sayfada birleştirir.

Responsive ve Erişilebilir Tasarım

Arayüz; 1024px, 720px ve 480px kesme noktalarıyla masaüstü, tablet ve mobil görünümelerde yatay taşma yaşamadan çalışır. Etiketlenmiş form alanları, `role="dialog" / aria-modal` özellikli modallar, görünür odak stilleri, atlama bağlantısı ve `aria-live` ile duyurulan durum bilgileri erişilebilirliği destekler.

KURULUM

Kurulum ve Çalıştırma

Gereksinimler

- Node.js 18 veya üzeri
- npm

Bağımlılıkların Kurulumu

```
cd taskflow-api  
npm install
```

Ortam Değişkenleri (İsteğe Bağlı)

Proje, `dotenv` ile `.env` dosyasından ortam değişkeni okur. Örnek `.env.example` dosyası `PORT=3000` değerini içerir. `.env` dosyası oluşturulmazsa uygulama varsayılan olarak `3000` portunu kullanır.

```
cp .env.example .env
```

Çalıştırma

Normal (üretim benzeri) çalıştırma:

```
npm start
```

Geliştirme sırasında dosya değişikliklerinde otomatik yeniden başlatma için:

```
npm run dev
```

Sunucu varsayılan olarak `3000` portunda başlar. Tarayıcıda `http://localhost:3000` adresine gidildiğinde TaskFlow web arayüzü açılır. API, temel bir yol öneki (`/api` gibi) kullanılmadan doğrudan aynı adres üzerinden (`http://localhost:3000/tasks` , `http://localhost:3000/reports/...`) kullanılabilir.

Proje Yapısı ve Veri Modeli

Kaynak kod, herkese açık bir GitHub deposunda yayınlanmıştır:

<https://github.com/mehmetozdmirrr/TaskFlow-Backend>

Klasör Yapısı

```
taskflow-api/
├── src/
│   ├── app.js           Express uygulaması ve route bağlantıları
│   ├── server.js        Portu okuyup sunucuyu başlatır
│   ├── routes/          tasks.js, reports.js
│   ├── middleware/      logger, validateTask, notFound, errorHandler
│   ├── controllers/     taskController, reportController
│   └── utils/           file.js, taskQuery.js
├── data/tasks.json      Kalıcı görev verisi
├── public/              index.html, css/, js/ (dashboard arayüzü)
└── docs/               Postman koleksiyonu, ekran görüntüleri
```

Task Veri Modeli

Alan	Zorunlu mu	Kural
id	—	Sunucu üretir (<code>Date.now()</code>) tabanlı, benzersizleştirilir
title	Evet	String, trim sonrası 3–100 karakter
description	Evet	String, trim sonrası 10–500 karakter
assignee	Evet	String, trim sonrası en az 2 karakter
priority	Evet	low , medium , high
status	Create'te opsiyonel, update'te zorunlu	pending , in-progress , completed
createdAt / updatedAt	—	Sunucu üretir ve yönetir

POST /tasks isteğinde status alanı gönderilmezse sunucu varsayılan olarak pending atar. status gönderilir ama geçersiz bir değer içerirse istek 400 Bad Request ile reddedilir. PUT /tasks/:id tam güncelleme olarak ele alınır; id ve createdAt istemciden alınmaz, mevcut kayıttan korunur.

API Rotaları

CRUD Endpoint'leri

Method	Rota	Açıklama	Başarı Kodu
POST	/tasks	Yeni görev oluşturur	201
GET	/tasks	Görevleri listeler (filtre/arama/sıralama/sayfalama)	200
GET	/tasks/:id	Tek görevi döndürür	200 / 404
PUT	/tasks/:id	Görevi tam olarak günceller	200 / 404
DELETE	/tasks/:id	Görevi siler	200 / 404

Gelişmiş Listeleme, Arama ve Raporlar

Method	Rota	Açıklama
GET	/tasks?status=&priority=	Duruma ve önceliğe göre filtreler
GET	/tasks?sort=createdAt&order=	Oluşturulma tarihine göre sıralar
GET	/tasks?page=&limit=	Sayfalar (varsayılan page=1, limit=10)
GET	/tasks/search?keyword=	Başlık/açıklamada anahtar kelime araması
GET	/tasks/assignee/:name	Çalışana göre listeler
GET	/reports/completed	Tamamlanan görev sayısı
GET	/reports/pending	Bekleyen görev sayısı
GET	/reports/summary	Toplam/bekleyen/devam eden/tamamlanan/yüksek öncelikli sayıları

Doğrulama ve Kalıcılık Davranışı

- POST / PUT hataları tek bir errors dizisinde toplanır ve 400 ile döner.
- Geçersiz sorgu parametreleri (status , priority , sort , order , page , limit , keyword) 400 döner.
- Bilinmeyen görev veya endpoint istekleri 404 döner.
- Bozuk JSON gövdesi sunucuyu çökertmeden 400 ile ele alınır.
- Rapor endpoint'leri her istekte güncel veriyi okur, hiçbir zaman dosyaya yazmaz.

Kullanıcı Arayüzü

Dashboard, REST API üzerindeki tüm işlevleri tek bir sayfada bir araya getirir; oluşturmadan raporlamaya kadar bütün akış görsel ve etkileşimli bir arayüzle desteklenir.

Panel ve Etkileşim

- Toplam, bekleyen, devam eden, tamamlanan ve yüksek öncelikli görev sayılarını gösteren 5 istatistik kartı
- Görev oluşturma ve düzenleme modalları
- Silme işlemi için ayrı bir onay modalı
- Durum ve öncelik rozetleriyle görev kartları
- Anahtar kelime araması ve çalışana göre filtre
- Durum/öncelik filtreleri ve 10/20/50 sayfa boyutu seçenekli sayfalama
- Artan/azalan sıralama seçici
- Başarılı/başarısız işlemler için toast bildirimleri

Durum Yönetimi

- Yükleniyor, boş sonuç ve hata durumları için ayrı paneller
- Sayfalama, toplam sayfa sayısı sıfır olduğunda otomatik gizlenir

Duyarlı Tasarım ve Erişilebilirlik

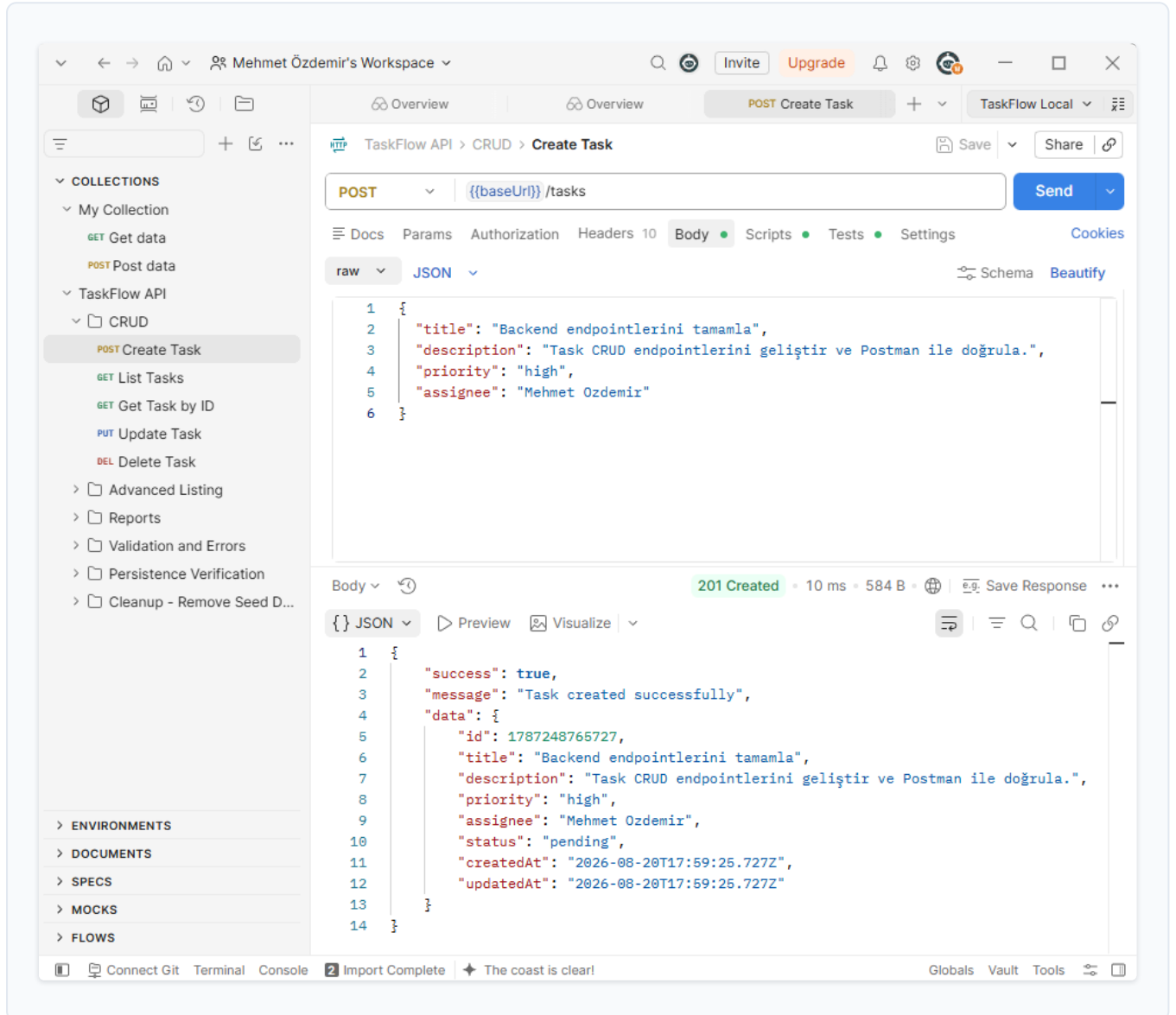
- 1024px, 720px ve 480px kesme noktalarıyla masaüstü, tablet ve mobil uyum
- Uzun başlık/açıklamalarda yatay taşma olmadan düzen korunur
- `label` ile ilişkilendirilmiş form alanları
- `role="dialog" / alertdialog` ve `aria-modal` ile modallar, `Escape` ile kapanır
- Görünür odak stilleri ve atlama bağlantısı (skip link)
- `prefers-reduced-motion` desteği
- Durum listesi ve sonuç sayacı `aria-live="polite"` ile duyurulur

Görev Oluşturma

POST /tasks

201 Created

Sunucu geçerli bir görev kaydı oluşturur ve id, createdAt, updatedAt alanları sunucu tarafından üretilmiş tam nesneyi döndürür.



Görevleri Listeleme

GET /tasks

200 OK

Kayıtlı tüm görevler, toplam kayıt sayısı, sayfa ve limit bilgisini içeren meta verisiyle birlikte döner.

The screenshot shows the Postman interface with a workspace named 'Mehmet Özdemir's Workspace'. The active collection is 'TaskFlow API' and the selected endpoint is 'GET List Tasks'. The request is a GET request to the URL 'http://{{baseUrl}}/tasks'. The response is a 200 OK status with a response time of 9 ms and a body size of 597 B. The response body is a JSON object:

```
{
  "success": true,
  "data": [
    {
      "id": 1787248765727,
      "title": "Backend endpointlerini tamamla",
      "description": "Task CRUD endpointlerini geliştir ve Postman ile doğrula.",
      "priority": "high",
      "assignee": "Mehmet Ozdemir",
      "status": "pending",
      "createdAt": "2026-08-20T17:59:25.727Z",
      "updatedAt": "2026-08-20T17:59:25.727Z"
    }
  ],
  "meta": {
    "total": 1,
    "page": 1,
    "limit": 10,
    "totalPages": 1
  }
}
```

Tek Görev Getirme

GET /tasks/:id

200 OK

Belirtilen id'ye sahip görev, tüm alanlarıyla eksiksiz biçimde döner.

The screenshot displays the Postman interface for a REST client. The workspace is named "Mehmet Özdemir's Workspace". The left sidebar shows a collection named "TaskFlow API" with a sub-collection "CRUD". The "Get Task by ID" endpoint is selected. The request is a GET method to the URL "http://localhost:3000/tasks/1787248765727". The response is a 200 OK status with a response time of 8 ms and a body size of 581 B. The response body is a JSON object:

```
{  "success": true,  "message": "Task retrieved successfully",  "data": {    "id": "1787248765727",    "title": "Backend endpointlerini tamamla",    "description": "Task CRUD endpointlerini geliştir ve Postman ile doğrula.",    "priority": "high",    "assignee": "Mehmet Ozdemir",    "status": "pending",    "createdAt": "2026-08-20T17:59:25.727Z",    "updatedAt": "2026-08-20T17:59:25.727Z"  }}}
```

The bottom status bar shows "Import Complete" and "Fix task id type assertion".

Görev Güncelleme

PUT /tasks/:id

200 OK

title, description, assignee, priority ve status alanları yeniden doğrulanarak güncellenir; id ve createdAt korunur, updatedAt yenilenir.

The screenshot displays the Postman interface for a PUT request to the endpoint `PUT /tasks/:id`. The request body is a JSON object with the following fields:

```
1 {
2   "title": "Backend endpointlerini tamamla ve doğrula",
3   "description": "Task CRUD endpointlerini geliştir, Postman ile doğrula ve sonucu kaydet.",
4   "priority": "medium",
5   "assignee": "Mehmet Ozdemir",
6   "status": "in-progress"
7 }
```

The response is a 200 OK status with a JSON body:

```
1 {
2   "success": true,
3   "message": "Task updated successfully",
4   "data": {
5     "id": 1787248765727,
6     "title": "Backend endpointlerini tamamla ve doğrula",
7     "description": "Task CRUD endpointlerini geliştir, Postman ile doğrula ve sonucu kaydet.",
8     "priority": "medium",
9     "assignee": "Mehmet Ozdemir",
10    "status": "in-progress",
11    "createdAt": "2026-08-20T17:59:25.727Z",
12    "updatedAt": "2026-08-20T18:11:19.104Z"
13  }
14 }
```

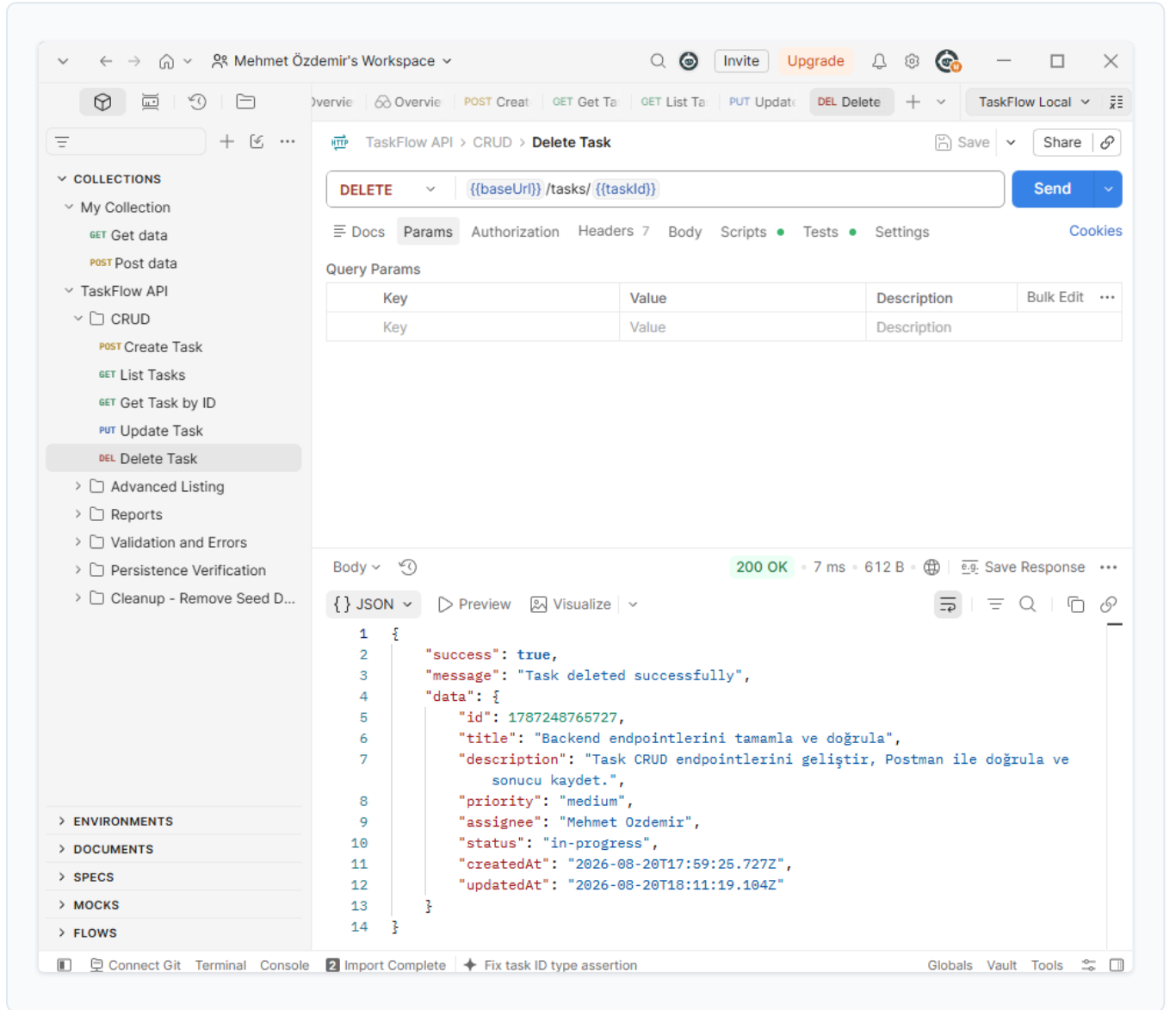
The interface also shows a sidebar with collections, environments, documents, specs, mocks, and flows. The bottom status bar indicates "Import Complete" and "The coast is clear!"

Görev Silme

DELETE /tasks/:id

200 OK

Görev kalıcı olarak kaldırılır ve silinen kaydın bilgisi cevap gövdesinde döner.



The screenshot shows the Postman interface for a DELETE API test. The test is named "Delete Task" and is located under the "CRUD" folder. The URL is "{{baseUrl}}/tasks/{{taskId}}". The response is a 200 OK status with a 7 ms response time and 612 B body. The response body is a JSON object containing success, message, and data fields.

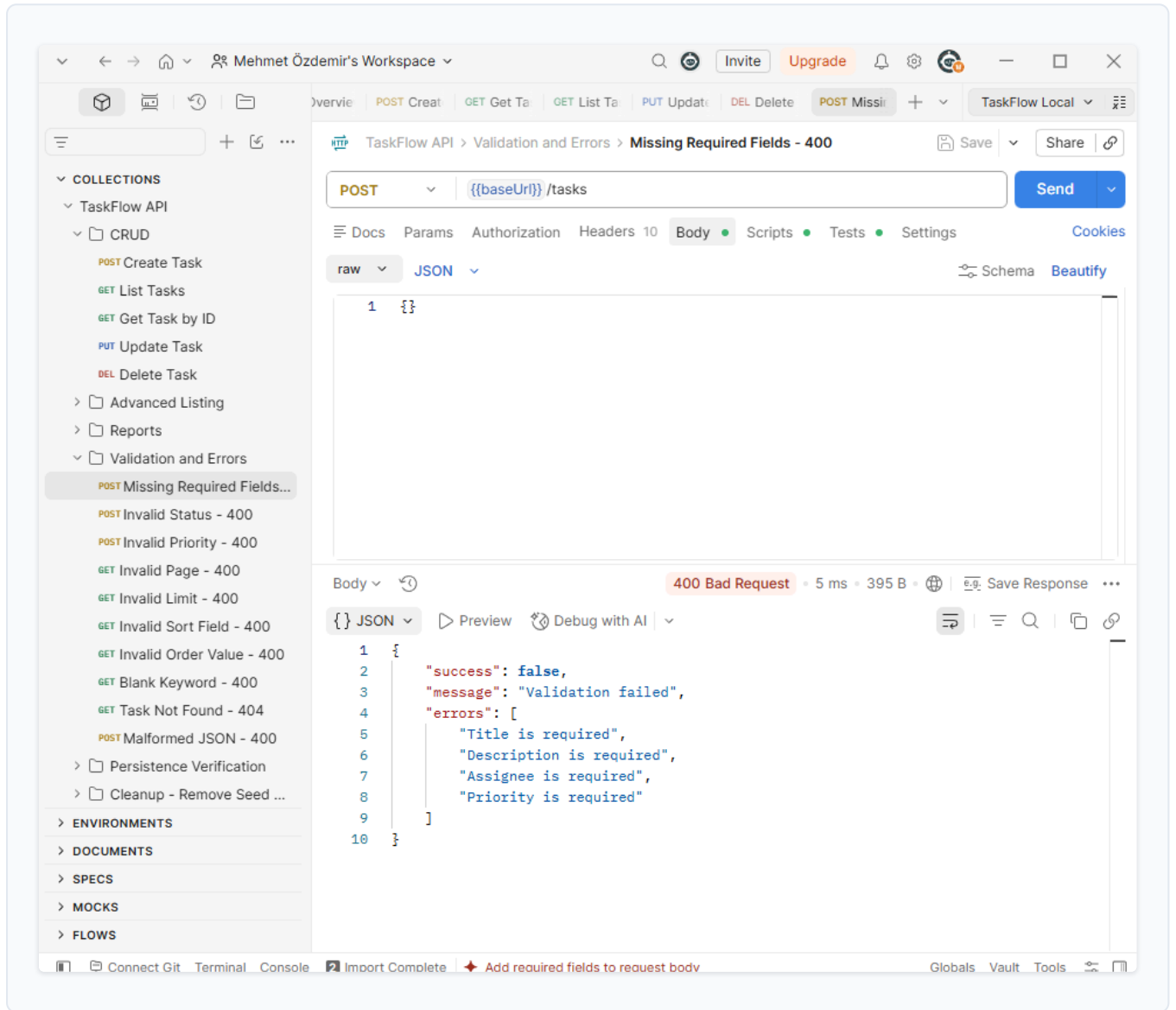
```
1 {
2   "success": true,
3   "message": "Task deleted successfully",
4   "data": {
5     "id": 1787248765727,
6     "title": "Backend endpointlerini tamamla ve doğrula",
7     "description": "Task CRUD endpointlerini geliştir, Postman ile doğrula ve sonucu kaydet.",
8     "priority": "medium",
9     "assignee": "Mehmet Ozdemir",
10    "status": "in-progress",
11    "createdAt": "2026-08-20T17:59:25.727Z",
12    "updatedAt": "2026-08-20T18:11:19.104Z"
13  }
14 }
```

Doğrulama Hatası

POST /tasks

400 Bad Request

Doğrulama ara katmanı eksik veya hatalı alanları tek bir errors dizisinde toplayarak isteği alan bazlı, açık mesajlarla reddeder.

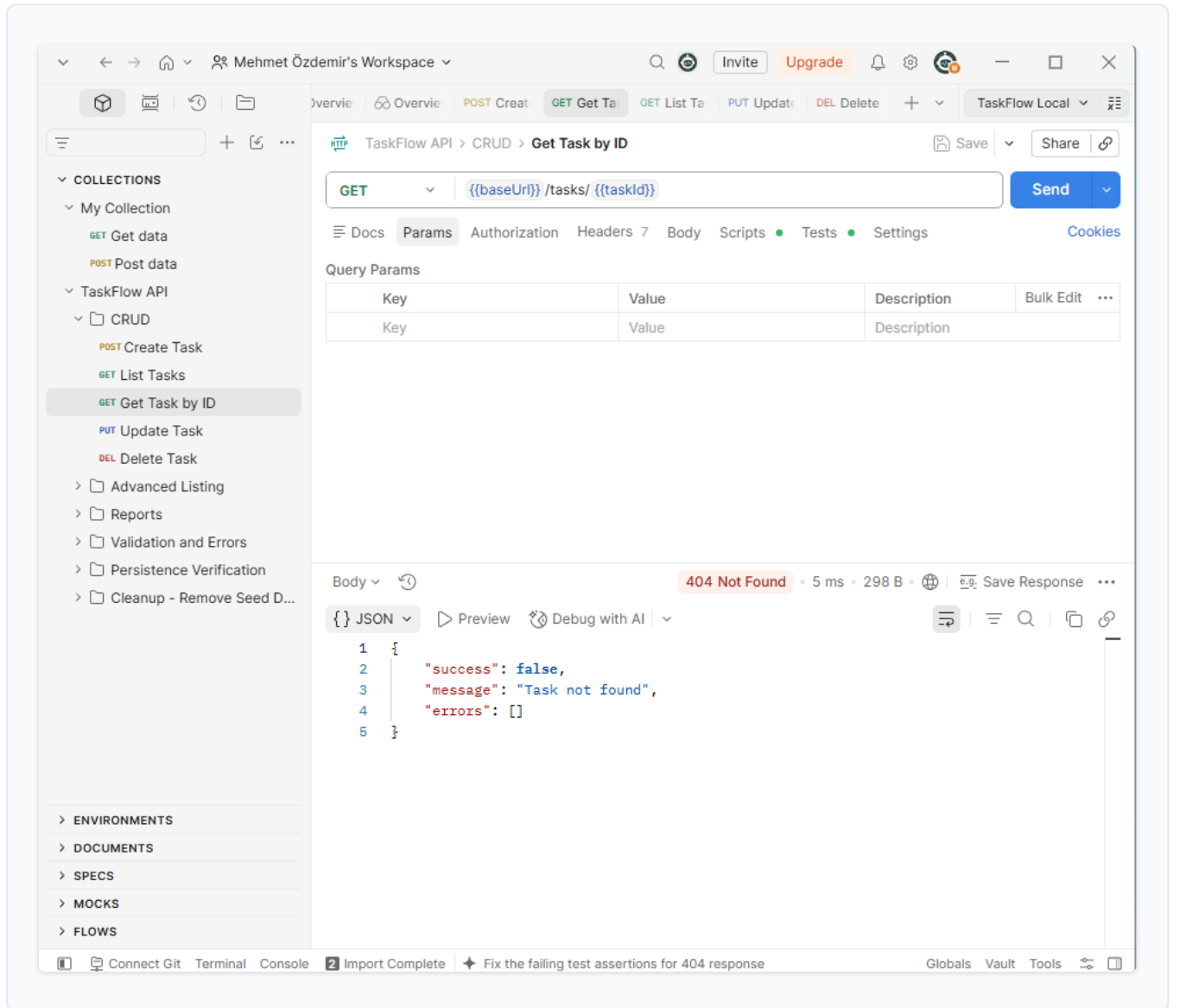


Görev Bulunamadı

GET /tasks/:id

404 Not Found

Var olmayan bir id için sunucu, bilinmeyen kaynağı doğru biçimde ele alarak net bir "Task not found" cevabı döner.

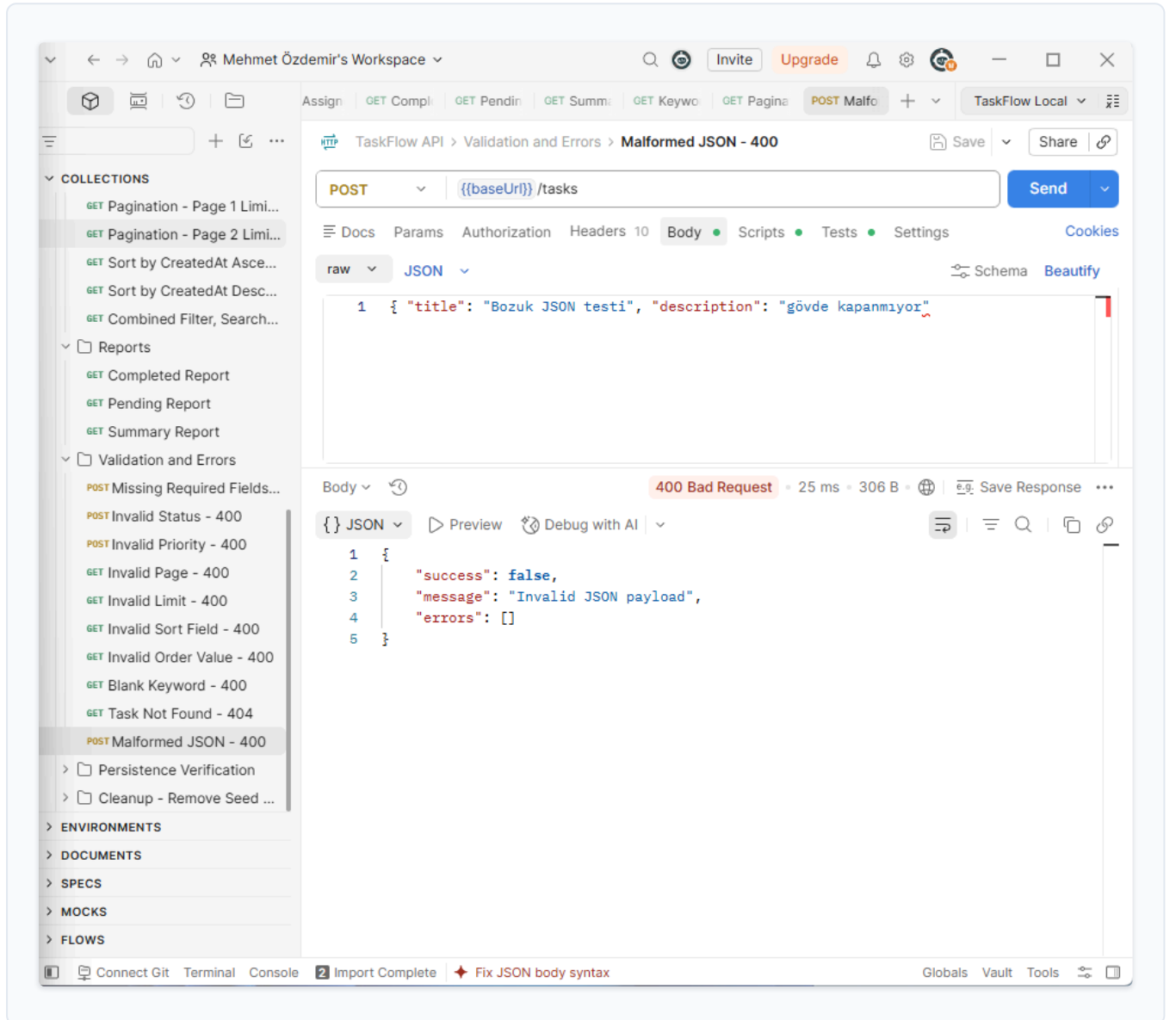


Bozuk JSON Gövdesi

POST /tasks

400 Bad Request

Express'in JSON ayrıştırıcısı hatalı biçimlendirilmiş gövdeyi güvenle yakalar; sunucu çökmeden "Invalid JSON payload" mesajıyla yanıt verir.



Birleşik Filtre

GET /tasks?status=...&priority=...

200 OK

status ve priority parametreleri birlikte uygulanarak sonuç kümesi daraltılır.

The screenshot displays the TaskFlow API client interface. The top navigation bar includes a search icon, a user profile icon, and buttons for 'Invite' and 'Upgrade'. The main workspace is titled 'Mehmet Özdemir's Workspace'. The left sidebar shows a list of collections, including 'POST Seed - Create Task 1 (...)', 'POST Seed - Create Task 2 (...)', 'POST Seed - Create Task 3 (...)', 'POST Seed - Create Task 4 (...)', 'POST Seed - Create Task 5 (...)', 'GET Filter by Status - pending', 'GET Filter by Priority - high', and 'GET Combined Status and Pr...'. The main panel shows a GET request to the endpoint `{{baseUrl}}/tasks?status=pending&priority=high`. The response is a 200 OK status with a response time of 6 ms and a body size of 595 B. The response body is a JSON object with the following structure:

```
{
  "success": true,
  "data": [
    {
      "id": 1787250189369,
      "title": "Backend API dokümantasyonu hazırla",
      "description": "Tüm endpointlerin istek ve cevap örneklerini yaz.",
      "priority": "high",
      "assignee": "Mehmet Ozdemir",
      "status": "pending",
      "createdAt": "2026-08-20T18:23:09.369Z",
      "updatedAt": "2026-08-20T18:23:09.369Z"
    }
  ],
  "meta": {
    "total": 1,
    "page": 1,
    "limit": 10,
    "totalPages": 1
  }
}
```


Çalışana Göre Listeleme

GET /tasks/assignee/:name

200 OK

Belirtilen çalışana atanmış görevler, büyük/küçük harf duyarlı eşleşmeyle listelenir.

The screenshot displays the TaskFlow API testing interface. The left sidebar shows a collection of API tests, with 'GET Assignee Listing - URL-Encoded Name' selected. The main panel shows the details of this test, including the URL, method, and response body. The response body is a JSON object containing a list of tasks assigned to Mehmet Özdemir.

```
{
  "success": true,
  "data": [
    {
      "id": 1787250189369,
      "title": "Backend API dokümantasyonu hazırla",
      "description": "Tüm endpointlerin istek ve cevap örneklerini yaz.",
      "priority": "high",
      "assignee": "Mehmet Ozdemir",
      "status": "pending",
      "createdAt": "2026-08-20T18:23:09.369Z",
      "updatedAt": "2026-08-20T18:23:09.369Z"
    },
    {
      "id": 1787250194556,
      "title": "Kullanıcı arayüzü responsive testleri",
      "description": "Mobil ve masaüstü görünümde taşma kontrolü yap.",
      "priority": "medium",
      "assignee": "Mehmet Ozdemir",
      "status": "in-progress",
      "createdAt": "2026-08-20T18:23:14.556Z",
      "updatedAt": "2026-08-20T18:23:14.556Z"
    }
  ],
  "meta": {
    "total": 2,
    "page": 1,
    "limit": 10,
    "totalPages": 1
  }
}
```

Anahtar Kelime Araması

GET /tasks/search?keyword=...

200 OK

title ve description alanlarında büyük/küçük harf duyarlı arama yapılarak eşleşen görevler döner.

The screenshot shows the TaskFlow API client interface. The left sidebar displays a collection of API endpoints under 'TaskFlow API' and 'Advanced Listing'. The main panel shows a GET request to the endpoint `GET {{baseUrl}} /tasks/search?keyword=rapor`. The response is a 200 OK status with a response time of 17 ms and a body size of 596 B. The response body is a JSON object:

```
{
  "success": true,
  "data": [
    {
      "id": 1787250204167,
      "title": "Rapor endpointlerini doğrula",
      "description": "Completed, pending ve summary sayılarını karşılaştır.",
      "priority": "high",
      "assignee": "Ayse Yilmaz",
      "status": "completed",
      "createdAt": "2026-08-20T18:23:24.167Z",
      "updatedAt": "2026-08-20T18:23:24.167Z"
    }
  ],
  "meta": {
    "total": 1,
    "page": 1,
    "limit": 10,
    "totalPages": 1
  }
}
```

Sayfalama

GET /tasks?page=2&limit=...

200 OK

page ve limit parametreleriyle sonuç kümesi sayfanır; meta verisi geçerli sayfayı ve toplam sayfa sayısını gösterir.

The screenshot displays the TaskFlow API testing interface. The left sidebar shows a list of collections, including 'Advanced Listing' and 'Reports'. The main area shows a GET request to the endpoint `/{baseUrl}/tasks?page=2&limit=2`. The response is a 200 OK status with a 7 ms response time and 871 B of data. The response body is a JSON object containing a list of tasks and a meta object.

Query Params

Key	Value	Description	Bulk Edit
page	2		
limit	2		

Body

```
{
  "tasks": [
    {
      "id": 1787250211587,
      "title": "Veritabanı yerine JSON dosyası kullan",
      "description": "fs modülü ile okuma yazma işlemlerini test et.",
      "priority": "low",
      "assignee": "Ayse Yilmaz",
      "status": "pending",
      "createdAt": "2026-08-20T18:23:31.587Z",
      "updatedAt": "2026-08-20T18:23:31.587Z"
    },
    {
      "id": 1787250211587,
      "title": "Veritabanı yerine JSON dosyası kullan",
      "description": "fs modülü ile okuma yazma işlemlerini test et.",
      "priority": "high",
      "assignee": "Ayse Yilmaz",
      "status": "completed",
      "createdAt": "2026-08-20T18:23:24.167Z",
      "updatedAt": "2026-08-20T18:23:24.167Z"
    }
  ],
  "meta": {
    "total": 5,
    "page": 2,
    "limit": 2,
    "totalPages": 3
  }
}
```

Tamamlanan Görev Raporu

GET /reports/completed

200 OK

status alanı completed olan görevlerin toplam sayısını döner.

The screenshot displays the TaskFlow API client interface. The top navigation bar includes a search icon, a user profile icon, and buttons for 'Invite' and 'Upgrade'. The left sidebar shows a list of collections under 'Mehmet Özdemir's Workspace', including 'POST Seed - Create Task 1 (...)', 'POST Seed - Create Task 2 (...)', 'POST Seed - Create Task 3 (A...', 'POST Seed - Create Task 4 (...)', 'POST Seed - Create Task 5 (...)', 'GET Filter by Status - pending', 'GET Filter by Priority - high', 'GET Combined Status and Pr...', 'GET Keyword Search - rapor', 'GET Keyword Search - No M...', 'GET Assignee Listing - Meh...', 'GET Assignee Listing - URL-...', 'GET Pagination - Page 1 Limi...', 'GET Pagination - Page 2 Limi...', 'GET Sort by CreatedAt Asce...', 'GET Sort by CreatedAt Desc...', 'GET Combined Filter, Search...', 'Reports', 'Completed Report', 'Pending Report', 'Summary Report', 'Validation and Errors', 'ENVIRONMENTS', 'DOCUMENTS', 'SPECS', 'MOCKS', and 'FLOWS'. The main panel shows the 'TaskFlow API > Reports > Completed Report' endpoint. The method is 'GET' and the URL is '{{baseUrl}} /reports/completed'. The response status is '200 OK' with a response time of '6 ms' and a size of '350 B'. The response body is JSON, showing a successful report generation with the following structure:

```
1 {
2   "success": true,
3   "message": "Completed tasks report generated successfully",
4   "data": {
5     "status": "completed",
6     "count": 2
7   }
8 }
```

The bottom status bar shows 'Connect Git', 'Terminal', 'Console', 'Import Complete', and 'The coast is clear!'. The right sidebar includes 'Globals', 'Vault', and 'Tools'.

Bekleyen Görev Raporu

GET /reports/pending

200 OK

Devam edenler hariç tutularak yalnızca pending durumundaki görevlerin sayısı döner.

The screenshot displays the TaskFlow API client interface. The top navigation bar includes a search icon, a user profile icon, and buttons for 'Invite' and 'Upgrade'. The main interface is divided into a left sidebar and a main content area. The sidebar contains a 'COLLECTIONS' section with a list of API endpoints, including 'GET Keyword Search - No M...', 'GET Assignee Listing - Meh...', 'GET Assignee Listing - URL-...', 'GET Pagination - Page 1 Limi...', 'GET Pagination - Page 2 Limi...', 'GET Sort by CreatedAt Asce...', 'GET Sort by CreatedAt Desc...', 'GET Combined Filter, Search...', 'GET Completed Report', 'GET Pending Report', 'GET Summary Report', 'POST Missing Required Fields...', 'POST Invalid Status - 400', 'POST Invalid Priority - 400', 'GET Invalid Page - 400', 'GET Invalid Limit - 400', 'GET Invalid Sort Field - 400', 'GET Invalid Order Value - 400', 'GET Blank Keyword - 400', and 'GET Task Not Found - 404'. The main content area shows a 'TaskFlow API > Reports > Pending Report' view. The request method is 'GET' and the URL is '{{baseUrl}}/reports/pending'. The response status is '200 OK' with a response time of '5 ms' and a response size of '346 B'. The response body is a JSON object:

```
{  "success": true,  "message": "Pending tasks report generated successfully",  "data": {    "status": "pending",    "count": 2  }}
```

Özet Raporu

GET /reports/summary

200 OK

Toplam, bekleyen, devam eden, tamamlanan ve yüksek öncelikli görev sayılarını tek bir yanıtta birleştirir.

The screenshot displays the TaskFlow API client interface. The left sidebar shows a collection of API endpoints under 'Reports', with 'GET Summary Report' selected. The main panel shows the request details for the 'GET /reports/summary' endpoint. The response is a 200 OK status with a 7 ms response time and 379 B body size. The response body is in JSON format, showing a successful report generation with the following data:

```
{  "success": true,  "message": "Summary report generated successfully",  "data": {    "total": 5,    "pending": 2,    "inProgress": 1,    "completed": 2,    "highPriority": 2  }}
```

Proje Özeti

TaskFlow, Express.js tabanlı bir REST API ile bu API üzerinde çalışan responsive bir web arayüzünü tek bir uygulamada birleştiren eksiksiz bir görev yönetim çözümüdür. Görev yaşam döngüsünün tamamı (oluşturma, listeleme, güncelleme, silme), gelişmiş listeleme ve raporlama yetenekleriyle birlikte sunulur.

Teknolojiler

- Node.js 18+ ve Express.js 5
- `dotenv` ile ortam yapılandırması
- `fs/promises` üzerinden JSON dosya kalıcılığı
- Vanilla HTML, CSS ve JavaScript frontend

Uygulanan Yetenekler

- Tam CRUD, filtreleme, arama, sıralama ve sayfalama
- Çalışana göre listeleme ve özet raporlar
- Merkezi doğrulama ve hata yönetimi
- Responsive ve erişilebilir dashboard arayüzü

Test Kapsamı

- `node --check` ile sözdizimi kontrolleri
- CRUD, gelişmiş listeleme, doğrulama ve kalıcılık için uçtan uca HTTP kontrolleri
- Masaüstü ve mobil tarayıcı kontrolleri
- 46 istek içeren Postman koleksiyonu ve 15 test görüntüsü

PROJE & GELİŞTİRİCİ

TaskFlow — Görev ve Proje Yönetim Sistemi

Mehmet Özdemir

GİTHUB DEPOSU

<https://github.com/mehmetozdmirrr/TaskFlow-Backend>